

A Note on Efficient Privacy-Preserving Similarity Search for Encrypted Vectors

Dongfang Zhao

University of Washington, USA
dzhao@cs.washington.edu

Abstract. Traditional approaches to vector similarity search over encrypted data rely on fully homomorphic encryption (FHE) to enable computation without decryption. However, the substantial computational overhead of FHE makes it impractical for large-scale real-time applications. This work explores a more efficient alternative: using additively homomorphic encryption (AHE) for privacy-preserving similarity search. We consider scenarios where either the query vector or the database vectors remain encrypted, a setting that frequently arises in applications such as confidential recommender systems and secure federated learning. While AHE only supports addition and scalar multiplication, we show that it is sufficient to compute inner product similarity—one of the most widely used similarity measures in vector retrieval. Compared to FHE-based solutions, our approach significantly reduces computational overhead by avoiding ciphertext-ciphertext multiplications and bootstrapping, while still preserving correctness and privacy. We present an efficient algorithm for encrypted similarity search under AHE and analyze its error growth and security implications. Our method provides a scalable and practical solution for privacy-preserving vector search in real-world machine learning applications.

1 Introduction

Vector similarity search is a fundamental operation in modern machine learning and information retrieval applications, particularly in vector databases [16] used for recommendation systems, retrieval-augmented generation (RAG), and large-scale search [20]. A key challenge in these applications is preserving data privacy while maintaining efficient query processing [19]. Fully Homomorphic Encryption (FHE) has been widely explored for encrypted similarity search but remains computationally expensive, making it impractical for real-time large-scale applications [9,4,8,6]. Open-source implementations such as Microsoft SEAL [18] and IBM HELib [10] provide FHE-based secure computation, but their high computational overhead significantly limits practical scalability [12].

In many privacy-sensitive applications [13,19], only one component—either the stored database vectors or the query vector—requires encryption, but not both. This occurs naturally in settings such as secure federated learning, confidential recommendation systems, and privacy-preserving search. The ability to

perform similarity computations in such settings without decrypting the data is critical for balancing privacy and efficiency. Existing secure computation frameworks, such as SecureML [15], provide strong privacy guarantees for machine learning tasks but often introduce significant computational overhead, making them less suitable for large-scale vector retrieval.

We investigate the feasibility of performing vector similarity search using only additively homomorphic encryption (AHE), which supports addition and scalar multiplication in the encrypted domain. Unlike FHE, which enables arbitrary computations at a high cost, AHE provides a significantly more efficient alternative by allowing direct computation of inner products without expensive ciphertext-ciphertext multiplications or bootstrapping. Despite its limited functionality, we demonstrate that inner product similarity—one of the most widely used metrics in vector retrieval—can still be efficiently computed under AHE, making it a viable choice for secure and scalable encrypted search.

This work shows that meaningful similarity search can be performed efficiently even when only addition is supported homomorphically. By leveraging AHE, we achieve substantial computational savings compared to traditional fully homomorphic methods while preserving correctness and privacy. These results suggest that lightweight homomorphic encryption schemes can play a crucial role in privacy-preserving vector search, providing a practical and scalable alternative to existing solutions.

2 Preliminaries and Related Work

2.1 Vector Databases

Vector databases have gained significant attention with the rise of applications such as natural language processing, recommendation systems, and information retrieval, where large-scale vector similarity searches are fundamental. Various systems have been designed to support these needs. For instance, Pinecone [1] and Vearch [2] provide efficient vector search capabilities but often lack flexibility in handling non-vector queries or supporting mixed workloads. These systems primarily rely on approximate nearest neighbor (ANN) search techniques, with graph-based methods such as Hierarchical Navigable Small Worlds (HNSW) [14] being widely adopted for scaling large datasets [11].

Among purpose-built vector database management systems, Milvus [20] distinguishes itself by supporting multiple index types and hybrid queries, allowing structured attributes to be combined with vector-based similarity searches. This flexibility enables Milvus to integrate traditional query optimization techniques with emerging data modalities, including unstructured text and images. Other systems, such as Weaviate [3] and SingleStore-V [5], further enhance vector search capabilities by incorporating features like predicate-based vector search and cost-based optimization, making them suitable for federated and distributed environments.

Despite these advancements, many existing solutions lack efficient mechanisms for preserving privacy in federated learning and distributed systems. This

limitation is particularly critical for real-time applications, where cryptographic protocols often introduce significant computational overhead [19]. While prior work in privacy-preserving machine learning, such as SecureML [15], establishes foundational approaches, our protocol aims to enhance vector data management by integrating secure and efficient cryptographic mechanisms. These improvements ensure both high-performance execution and strong privacy guarantees in federated environments.

2.2 Similarity Metrics

Vector similarity measures play a crucial role in applications such as retrieval-augmented generation, recommendation systems, and large-scale search. Various similarity metrics have been developed, each tailored to different types of data distributions and retrieval objectives.

Cosine Similarity One of the most widely used similarity metrics, cosine similarity measures the cosine of the angle between two vectors. Given two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, it is computed as:

$$\text{CosSim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}.$$

Cosine similarity is invariant to the magnitude of vectors, making it particularly effective for text embeddings and high-dimensional spaces where relative orientation is more informative than magnitude.

Inner Product Inner product similarity directly measures the dot product between two vectors:

$$\text{InnerProd}(\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^d x_i y_i.$$

This measure is commonly used in machine learning models and recommendation systems, where higher values indicate greater relevance or similarity. It is particularly useful in neural network-based retrieval tasks.

Euclidean Distance Euclidean distance quantifies the straight-line distance between two vectors:

$$\text{EucDist}(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\| = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}.$$

Unlike cosine similarity, Euclidean distance is sensitive to vector magnitude and is often preferred in metric spaces where absolute differences matter, such as in image retrieval or clustering applications.

Learned Similarity Metrics Recent advances in similarity search have explored learned similarity functions that adapt retrieval behavior beyond traditional metrics. The work by Ding et al. [7] introduces a framework that learns task-specific similarity metrics to improve retrieval performance. Unlike predefined measures such as cosine or Euclidean distance, their approach optimizes a similarity function through machine learning, enabling more adaptive and context-aware retrieval in large-scale vector search.

2.3 Algebraic Homomorphism

The notion *homomorphism* refers to a class of functions that preserve the algebraic structures of the input and output spaces. More specifically, an algebraic group¹ can be relabelled and transformed, through a homomorphic function, into another algebraic group without changing the relationship among the elements. An algebraic *group* is defined a nonempty set of elements along with a binary operator satisfying the closure, associativity, identity, and inverse properties.

Formally, a group G over a set S is represented as a tuple $(G, \oplus)$, where $\oplus$ is a binary operator. This operator adheres to four fundamental axioms or properties, expressed through first-order logical formulas: (i) For all $g, h \in S$, $g \oplus h \in S$; (ii) There exists a unique element $u \in S$ such that for all $g \in S$, $(g \oplus u = g)$ and $(u \oplus g = g)$; (iii) For every $g \in S$, there exists an element $h \in S$ such that $(g \oplus h = u)$ and $(h \oplus g = u)$, where h is often denoted as $-g$; and (iv) For all $g, h, j \in S$, $(g \oplus h) \oplus j = g \oplus (h \oplus j)$. Given another group $(H, \otimes)$ and a function $\varphi : G \rightarrow H$ that for all $g_1, g_2 \in G$ satisfies $\varphi(g_1) \otimes \varphi(g_2) = \varphi(g_1 \oplus g_2)$, we call function φ a homomorphism from G to H . This implies that φ preserves the group operation between the elements of G when mapped to the corresponding elements in H .

3 Method

3.1 Definitions and Notations

Given a plaintext vector $\mathbf{x} = (x_1, x_2, \dots, x_d)$ and an encrypted vector $\mathbf{y}' = Enc(\mathbf{y}) = (Enc(y_1), Enc(y_2), \dots, Enc(y_d))$, where each y_i is encrypted under an additively homomorphic cryptosystem, our goal is to compute an encrypted form of their similarity (i.e., inner product) s' :

$$s' \equiv Enc \left(\sum_{i=1}^d x_i y_i \right)$$

up to encryption randomness without revealing $\mathbf{y}$, such that $Dec(s') = s = \sum_{i=1}^d x_i y_i$.

Let's define an additively homomorphic encryption scheme formally:

¹ We use *algebraic group* to refer to a group structure in group theory, *not* the solutions to a system of polynomial equations in algebraic geometry.

Definition 1 (Additively Homomorphic Encryption). *An encryption scheme is said to be additively homomorphic if for any two plaintexts $a, b \in \mathbb{M}$ (the message space), and their respective encryptions $Enc(a), Enc(b) \in \mathbb{C}$ (the ciphertext space) under the public key, there exists a group operation $\odot$ in $\mathbb{C}$ such that:*

$$Dec(Enc(a) \odot Enc(b)) = a + b$$

where Dec is the decryption function, and ‘+’ is the addition operation in the plaintext space $\mathbb{M}$.

3.2 Homomorphic Computation of Inner Product

The homomorphic computation of the inner product can be described as follows. Let Enc denote the encryption function with $\odot$ as the operation in the ciphertext space, which may not be standard multiplication but rather a specific group operation defined for the encryption scheme. For some schemes, such as Pailliar [17], the $\odot$ operation is indeed arithmetic multiplication.

The encrypted similarity, $Sim(\cdot, \cdot)$ of vectors $\mathbf{x}$ and $Enc(\mathbf{y}) = (Enc(y_i))$ can be computed as:

$$Sim(\mathbf{x}, Enc(\mathbf{y})) = Enc\left(\sum_{i=1}^d x_i y_i\right) = \bigotimes_{i=1}^d Enc(y_i)^{x_i},$$

where $\bigotimes$ denotes the multiplication of the factors and the exponentiation is realized by the multiplication of the base ciphertexts. That is,

$$Enc(y_i)^{x_i} = \bigotimes_{j=1}^{x_i} Enc(y_i).$$

Combining the above two equations, we have

$$Sim(\mathbf{x}, Enc(\mathbf{y})) = \bigotimes_{i=1}^d \bigotimes_{j=1}^{x_i} Enc(y_i), \quad (1)$$

which allows us to compute the encrypted similarity between a pair of plaintext-ciphertext vectors.

3.3 Correctness of the Homomorphic Inner Product Computation

We prove that the encrypted similarity computation preserves the correctness of the inner product under decryption. Given a plaintext vector $\mathbf{x} = (x_1, x_2, \dots, x_d)$ and an encrypted vector $Enc(\mathbf{y}) = (Enc(y_1), Enc(y_2), \dots, Enc(y_d))$, the encrypted similarity is computed as:

$$Sim(\mathbf{x}, Enc(\mathbf{y})) = \bigotimes_{i=1}^d \bigotimes_{j=1}^{x_i} Enc(y_i). \quad (2)$$

Using the plaintext-ciphertext multiplication property of the encryption scheme, we have:

$$Enc(y_i)^{x_i} = \bigotimes_{j=1}^{x_i} Enc(y_i) = Enc(x_i y_i). \quad (3)$$

Applying this property to the inner product computation:

$$Sim(\mathbf{x}, Enc(\mathbf{y})) = \bigotimes_{i=1}^d Enc(y_i)^{x_i} = \bigotimes_{i=1}^d Enc(x_i y_i). \quad (4)$$

Since the encryption function Enc is additively homomorphic, we use the homomorphic addition property:

$$\bigotimes_{i=1}^d Enc(x_i y_i) = Enc\left(\sum_{i=1}^d x_i y_i\right). \quad (5)$$

Applying the decryption function Dec , we obtain:

$$Dec(Sim(\mathbf{x}, Enc(\mathbf{y}))) = Dec\left(Enc\left(\sum_{i=1}^d x_i y_i\right)\right). \quad (6)$$

Since decryption correctly recovers the original plaintext:

$$Dec\left(Enc\left(\sum_{i=1}^d x_i y_i\right)\right) = \sum_{i=1}^d x_i y_i. \quad (7)$$

Thus, we have proven that the homomorphic inner product computation preserves correctness:

$$Dec(Sim(\mathbf{x}, Enc(\mathbf{y}))) = \sum_{i=1}^d x_i y_i. \quad (8)$$

This demonstrates that the proposed method allows secure computation of the encrypted similarity while ensuring correctness upon decryption.

3.4 Security Analysis

We analyze the security of the proposed homomorphic inner product computation under the standard semantic security model. Specifically, we show that if the underlying encryption scheme is semantically secure under chosen plaintext attacks (IND-CPA), then the encrypted similarity computation does not reveal any information about the encrypted vector $\mathbf{y}$.

Threat Model We assume an adversary who observes all computations performed during the similarity computation process, including:

- The plaintext vector $\mathbf{x}$, which is publicly known.
- The set of ciphertexts $\{Enc(y_1), Enc(y_2), \dots, Enc(y_d)\}$.
- The final encrypted similarity result $Enc(s)$, where $s = \sum_{i=1}^d x_i y_i$.

The adversary's goal is to extract information about the encrypted values $\mathbf{y} = (y_1, y_2, \dots, y_d)$ beyond what is trivially deducible from $\mathbf{x}$ and $Enc(s)$.

Reduction to the Semantic Security of the Encryption Scheme We prove security by reduction: if an adversary $\mathcal{A}$ can break the privacy of $\mathbf{y}$ in our similarity computation scheme, then we can construct a simulator $\mathcal{S}$ that uses $\mathcal{A}$ as a subroutine to break the semantic security (IND-CPA) of the underlying encryption scheme.

1. Assume $\mathcal{A}$ can infer partial or full information about the encrypted values y_i from observing the computation of $Enc(s)$.
2. We construct $\mathcal{S}$ that interacts with an encryption oracle of an IND-CPA secure encryption scheme. The simulator $\mathcal{S}$:
 - Chooses a random plaintext vector $\mathbf{x}$.
 - Queries the encryption oracle to obtain $Enc(y_1), \dots, Enc(y_d)$ for randomly chosen y_i values.
 - Computes $Enc(s) = \bigotimes_{i=1}^d Enc(y_i)^{x_i}$.
3. If $\mathcal{A}$ can successfully extract information about any y_i , then $\mathcal{S}$ can use this to distinguish between encryptions of different messages, violating the IND-CPA security of the underlying encryption scheme.

Since the encryption scheme is assumed to be IND-CPA secure, no polynomial-time adversary can extract any meaningful information about y_i from its ciphertext $Enc(y_i)$. This ensures that our similarity computation process does not reveal additional information beyond what is trivially deducible from $\mathbf{x}$ and $Enc(s)$.

3.5 Computational Complexity Analysis

We analyze the computational complexity of the privacy-preserving inner product computation.

The total complexity consists of three main components: homomorphic multiplications (plaintext-ciphertext multiplications, PCM), homomorphic additions, and a final decryption step. Let N denote the security parameter of the encryption scheme, typically corresponding to the bit-length of the modulus. From the inner product computation formula:

$$Sim(\mathbf{x}, Enc(\mathbf{y})) = \bigotimes_{i=1}^d \bigotimes_{j=1}^{x_i} Enc(y_i), \quad (9)$$

we derive the following computational costs:

- **Plaintext-Ciphertext Multiplication (PCM) Cost:** Each encrypted value $Enc(y_i)$ is exponentiated by x_i , requiring $\mathcal{O}(\log x_i)$ modular multiplications using square-and-multiply exponentiation. For all d dimensions, the total cost is $\mathcal{O}(d \cdot T_{PCM})$.
- **Homomorphic Addition Cost:** Since we perform $d - 1$ additions in the encrypted domain, the total cost is $\mathcal{O}(d \cdot T_{Add})$.
- **Decryption Cost:** Only one final decryption is performed, contributing $\mathcal{O}(T_{Dec})$.

Thus, the overall computational complexity is:

$$\mathcal{O}(d \cdot (T_{PCM} + T_{Add}) + T_{Dec}). \quad (10)$$

For encryption schemes supporting additive homomorphism, we estimate:

$$\begin{aligned} T_{PCM} &= \mathcal{O}(\log x_i \cdot \log^2 N), \\ T_{Add} &= \mathcal{O}(1), \\ T_{Dec} &= \mathcal{O}(\log^2 N). \end{aligned}$$

Thus, the total complexity simplifies to:

$$\mathcal{O}(d \cdot \log^2 N). \quad (11)$$

Compared to fully homomorphic encryption (FHE), which typically requires $\mathcal{O}(\text{poly}(\log N))$ per operation due to bootstrapping overhead, this approach is significantly more efficient, making it suitable for real-time privacy-preserving similarity computation.

3.6 Error Growth in Homomorphic Inner Product Computation

In homomorphic encryption schemes, noise accumulates with each operation, impacting the correctness of the decryption process. For additively homomorphic encryption (AHE), the primary operations involved in computing an encrypted inner product are homomorphic additions and plaintext-ciphertext multiplications. We analyze the error growth in this context and discuss its implications for practical encrypted similarity search.

Noise Growth in Addition. Let $Enc(y_i)$ denote the encryption of a plaintext value y_i , and assume the encryption scheme introduces an initial noise term e_i such that:

$$Enc(y_i) = y_i + e_i \pmod{q}.$$

For an encrypted inner product:

$$Enc(s) = \sum_{i=1}^d x_i \cdot Enc(y_i),$$

the resulting noise term is:

$$e_s = \sum_{i=1}^d x_i e_i.$$

If e_i follows a uniform distribution $U(-B, B)$, then the variance of e_s is:

$$\text{Var}(e_s) = \sum_{i=1}^d x_i^2 \cdot \text{Var}(e_i) = dB^2 \mathbb{E}[x_i^2].$$

Thus, the noise grows linearly with dimension d , which is manageable for moderate dimensions but may require parameter tuning for high-dimensional vectors.

Asymptotic Noise Growth Bound. For a worst-case analysis, assuming x_i is bounded by X , we obtain:

$$|e_s| \leq dXB.$$

If X and B are constants, the noise growth remains $O(d)$, ensuring scalability for practical applications. However, for large plaintext values X , the noise can accumulate rapidly, requiring modulus switching or rescaling to maintain decryption accuracy.

Comparison to FHE. Unlike fully homomorphic encryption (FHE), where noise grows exponentially due to ciphertext-ciphertext multiplications, AHE only experiences linear noise growth. This eliminates the need for bootstrapping, significantly reducing computation cost. Instead, AHE can use modulus switching or rescaling to maintain correctness over multiple operations while keeping the noise under control.

4 Conclusion

This work investigates the feasibility of performing privacy-preserving similarity search using additively homomorphic encryption (AHE). While fully homomorphic encryption (FHE) enables arbitrary computations over encrypted data, its high computational overhead limits its practicality in large-scale applications. In contrast, AHE provides a lightweight alternative, supporting addition and scalar multiplication while significantly reducing computational costs.

We show that inner product similarity, a fundamental metric in vector retrieval, can be efficiently computed under AHE while maintaining privacy and correctness. Our approach is particularly well-suited for privacy-sensitive applications where only one component (either the database or the query vector) needs encryption, such as secure federated learning and confidential recommendation systems.

Future work will explore optimizations for specific encryption schemes, potential extensions to other similarity metrics, and trade-offs between efficiency and security in real-world deployments.

References

1. Pinecone: A vector database for machine learning. <https://www.pinecone.io/> (2025)
2. Vearch: A distributed system for efficient vector-based search. <https://github.com/vearch/vearch> (2025)
3. Weaviate: Graph-based vector database (2025), <https://weaviate.io/>
4. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. In: The 3rd Innovations in Theoretical Computer Science Conference (2012). <https://doi.org/10.1145/2090236.2090262>
5. Chen, C., Jin, C., Zhang, Y., Podolsky, S., Wu, C., Wang, S.P., Hanson, E., Sun, Z., Walzer, R., Wang, J.: Singlestore-v: An integrated vector database system in singlestore. *Proc. VLDB Endow.* **17**(12), 3772–3785 (Aug 2024)
6. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. In: Takagi, T., Peyrin, T. (eds.) 23rd International Conference on the Theory and Applications of Cryptology and Information Security (AsiaCrypt). Springer (2017)
7. Ding, B., Zhai, J.: Efficient retrieval with learned similarities. *Proceedings of the Web Conference (WWW)* (2025)
8. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive, Paper 2012/144* (2012), <https://eprint.iacr.org/2012/144>, <https://eprint.iacr.org/2012/144>
9. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: *Proceedings of the Forty-first Annual ACM Symposium on Theory of Computing (STOC)* (2009)
10. HELib: <https://github.com/homenc/HELlib> (Accessed 2022)
11. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with gpus. In: *IEEE Trans. Big Data.* pp. 535–547 (2019)
12. Khan, M.J., Fang, B., Cimino, G., Cirillo, S., Yang, L., Zhao, D.: Privacy-preserving artificial intelligence on edge devices: A homomorphic encryption approach. In: *2024 IEEE International Conference on Web Services (ICWS)*. pp. 395–405 (2024). <https://doi.org/10.1109/ICWS62655.2024.00061>
13. Li, Y., Ghosh, D., Gupta, P., Mehrotra, S., Panwar, N., Sharma, S.: Prism: Private verifiable set computation over multi-owner outsourced databases. In: *Proceedings of the 2021 International Conference on Management of Data.* p. 1116–1128. SIGMOD ’21, Association for Computing Machinery, New York, NY, USA (2021)
14. Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **42**(4), 824–836 (2020)
15. Mohassel, P., Zhang, Y.: Secureml: A system for scalable privacy-preserving machine learning. In: *2017 IEEE Symposium on Security and Privacy (SP)*. pp. 19–38. IEEE (2017)
16. Monir, S.S., Zhao, D.: Efficient feature extraction for image analysis through adaptive caching in vector databases. In: *2024 7th International Conference on Information and Computer Technologies (ICICT)*. pp. 193–198 (2024). <https://doi.org/10.1109/ICICT62343.2024.00036>
17. Paillier, P.: Public-key cryptosystems based on composite degree residuosity classes. In: *Proceedings of the 17th International Conference on Theory and Application of Cryptographic Techniques.* p. 223–238. EUROCRYPT’99, Springer-Verlag, Berlin, Heidelberg (1999)

18. Microsoft SEAL (release 3.7). <https://github.com/Microsoft/SEAL> (Sep 2021), microsoft Research, Redmond, WA.
19. T. Tawose, O., Dai, J., Yang, L., Zhao, D.: Toward efficient homomorphic encryption for outsourced databases through parallel caching. Proceedings of the ACM on Management of Data (SIGMOD) (May 2023)
20. Wang, J., et al.: Milvus: A purpose-built vector data management system. In: SIGMOD. pp. 2614–2627 (2021)